



Advanced JavaScript Handbook

A Complete Study Guide for Aspiring JavaScript Developers

Target Audience: You know basic JavaScript (variables, loops, arrays, objects, functions) and want to master the advanced concepts that professionals use every day.

Table of Contents

- 1. [Part 1 — JavaScript Execution Fundamentals](#)
- 2. [Part 2 — The "this" Keyword and Function Methods](#)
- 3. [Part 3 — Asynchronous JavaScript](#)
- 4. [Part 4 — Error Handling](#)
- 5. [Part 5 — Advanced Function Concepts](#)
- 6. [Part 6 — Special JavaScript Patterns](#)
- 7. [Part 7 — Prototypes and Object Internals](#)
- 8. [Part 8 — Modern JavaScript \(ES6+\)](#)
- 9. [Part 9 — Real World Examples](#)
- 10. [Part 10 — Common JavaScript Mistakes](#)
- 11. [Part 11 — Practice Section](#)

Part 1 — JavaScript Execution Fundamentals

1.1 How JavaScript Runs

JavaScript is an **interpreted** (or just-in-time compiled) programming language. When you write JavaScript code, it doesn't talk directly to your computer's hardware. Instead, a **JavaScript engine** reads and executes your code.

Real-World Analogy: Think of a JavaScript engine like a translator at the United Nations. You (the developer) write instructions in JavaScript. The engine translates those instructions into something the computer can understand and act on, line by line.

Popular JavaScript Engines:

Browser	Engine
Google Chrome	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore
Edge	V8

Browser	Engine
Node.js	V8

When you open a web page, the browser downloads the JavaScript file, hands it to the engine, and the engine:

1. **Parses** the code (reads it and checks for errors)
2. **Compiles** it to machine code (just-in-time compilation)
3. **Executes** the compiled code

1.2 Single-Threaded Nature

JavaScript is **single-threaded**, which means it can only do **one thing at a time**.

Real-World Analogy: Imagine a restaurant with only **one chef**. That chef can only cook one dish at a time. If someone orders a steak while the chef is making pasta, the steak has to **wait**. JavaScript works the same way — it processes tasks one after another, in order.

```
console.log("Task 1: Wash vegetables");
console.log("Task 2: Chop onions");
console.log("Task 3: Heat the pan");
```

Output:

```
Task 1: Wash vegetables
Task 2: Chop onions
Task 3: Heat the pan
```

Each task runs only after the previous one finishes. This is called **synchronous execution**.

Key Takeaway: JavaScript has one call stack and one memory heap. It executes code from top to bottom, one statement at a time.

1.3 The Call Stack

The **Call Stack** is a data structure that keeps track of **which function is currently running** and **what to run next**.

Real-World Analogy: Think of a stack of plates in a cafeteria. You can only add a plate on **top** and remove the plate on **top**. The Call Stack works the same way — the last function that was called is the first one to finish (Last In, First Out — **LIFO**).

```
function greet() {
  console.log("Hello!");
}
```

```
}

function askName() {
  console.log("What is your name?");
  greet();
}

askName();
```

Step-by-step Call Stack Visualization:

```
Step 1: askName() is called    → Stack: [askName]
Step 2: console.log() runs    → Stack: [askName, console.log]
Step 3: console.log() finishes → Stack: [askName]
Step 4: greet() is called     → Stack: [askName, greet]
Step 5: console.log() runs    → Stack: [askName, greet, console.log]
Step 6: console.log() finishes → Stack: [askName, greet]
Step 7: greet() finishes      → Stack: [askName]
Step 8: askName() finishes    → Stack: [] (empty)
```

Output:

```
What is your name?
Hello!
```

Stack Overflow: If a function calls itself endlessly, the stack fills up and crashes:

```
function forever() {
  forever(); // calls itself infinitely
}
forever(); // ✗ RangeError: Maximum call stack size exceeded
```

1.4 Execution Context

An **Execution Context** is an environment where JavaScript code is evaluated and executed. Think of it as a **box** that holds everything needed to run a piece of code.

Real-World Analogy: When you walk into a classroom, there's a specific environment — a whiteboard, desks, a teacher, textbooks. That classroom is the "execution context" for learning. Similarly, every time JavaScript runs code, it creates an environment with variables, functions, and the value of **this**.

Every execution context has **two phases**:

1. **Creation Phase** — JavaScript scans the code and sets up memory for variables and functions
2. **Execution Phase** — JavaScript runs the code line by line

1.4.1 Global Execution Context (GEC)

The **Global Execution Context** is created when your JavaScript file **first starts running**. There's always exactly **one** GEC.

```
var name = "Alice";

function sayHello() {
  console.log("Hello, " + name);
}

sayHello();
```

Creation Phase of GEC:

```
globalContext = {
  name: undefined,      // var is hoisted with value undefined
  sayHello: function() {}, // function is hoisted with full definition
  this: window          // (or 'global' in Node.js)
}
```

Execution Phase of GEC:

```
name = "Alice"           // now assigned the actual value
sayHello()               // function is called
```

1.4.2 Function Execution Context (FEC)

Every time a function is **called**, a **new** execution context is created for that function.

```
var color = "blue";

function paintWall(room) {
  var message = "Painting " + room + " " + color;
  console.log(message);
}

paintWall("bedroom");
paintWall("kitchen");
```

What happens:

- GEC is created → `color` and `paintWall` are set up

- `paintWall("bedroom")` → **new FEC created** for this call
 - `room = "bedroom"`, `message` is created
 - After `console.log`, this FEC is **destroyed**
 - `paintWall("kitchen")` → **another new FEC created**
 - `room = "kitchen"`, `message` is created
 - After `console.log`, this FEC is also **destroyed**
-

1.5 Hoisting

Hoisting is JavaScript's behavior of moving **declarations** to the top of their scope during the creation phase, **before** the code runs.

Real-World Analogy: Imagine a teacher who reads the entire class roster before the class starts. Even though Roll Call happens later, the teacher already **knows** all the names. Similarly, JavaScript "reads" all declarations before executing any code.

var Hoisting

```
console.log(age); // undefined (not an error!)
var age = 25;
console.log(age); // 25
```

Why? During the creation phase, JavaScript sees `var age` and reserves memory for it with the value `undefined`. The actual assignment `age = 25` happens during execution.

Function Hoisting

```
sayHi(); // "Hi there!" – works even before the function is defined!

function sayHi() {
  console.log("Hi there!");
}
```

Function declarations are fully hoisted — both the name and the body are available.

let and const — NOT Hoisted the Same Way

```
console.log(score); // ✗ ReferenceError: Cannot access 'score' before
initialization
let score = 100;
```

`let` and `const` are hoisted but placed in the **Temporal Dead Zone** (TDZ).

1.6 Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between the start of a scope and the point where a `let` or `const` variable is declared. During this period, the variable **exists** but **cannot be accessed**.

Real-World Analogy: You've ordered a package online. The package exists (it's been created at the warehouse), but you **can't use it** until it's delivered to your door. The time between ordering and delivery is the "Temporal Dead Zone."

```
{
  // TDZ for 'username' starts here
  console.log(username); // ✗ ReferenceError
  // TDZ continues...
  let username = "Bob"; // TDZ ends here
  console.log(username); // ☑ "Bob"
}
```

Why does TDZ exist? It helps catch bugs. If you try to use a variable before it's properly declared, JavaScript tells you immediately rather than giving you a confusing `undefined`.

1.7 Scope

Scope determines **where variables can be accessed** in your code.

Real-World Analogy: Think of scope like rooms in a house. A TV in the living room can only be used in the living room. A TV in the bedroom can only be used in the bedroom. But the WiFi router in the hallway can be used by **every room** — that's like a global variable.

Types of Scope

1. Global Scope — Variables declared outside any function or block.

```
var globalVar = "I'm everywhere!";

function test() {
  console.log(globalVar); // ☑ Accessible
}

test();
console.log(globalVar); // ☑ Accessible
```

2. Function Scope — Variables declared inside a function.

```
function myFunction() {
  var secret = "hidden";
  console.log(secret); // ☑ Accessible inside
```

```
}

myFunction();
console.log(secret); // ✗ ReferenceError – not accessible outside
```

3. Block Scope — Variables declared with `let` or `const` inside `{}`.

```
if (true) {
  let blockVar = "I'm block-scoped";
  console.log(blockVar); // ☑ Accessible inside the block
}

console.log(blockVar); // ✗ ReferenceError – not accessible outside
```

Important: `var` does NOT have block scope. Only `let` and `const` do.

1.8 Lexical Scope

Lexical Scope means that a function's scope is determined by **where it is written** in the code, not where it is called.

Real-World Analogy: Imagine you're a child in a family. You can access things in your room AND in the shared family living room. Your scope (access) is determined by where you **live** (where the function is written), not where you **visit**.

```
function outer() {
  let outerVar = "I belong to outer";

  function inner() {
    console.log(outerVar); // ☑ Can access parent's variable
  }

  inner();
}

outer(); // "I belong to outer"
```

Step-by-step:

1. `outer()` is called — creates its own scope with `outerVar`
2. `inner()` is defined **inside** `outer` — so `inner` can "see" `outer`'s variables
3. When `inner()` looks for `outerVar`, it first checks its own scope (not found), then looks in the parent scope (found!)

This process of looking up the scope chain is called the **Scope Chain**.

1.9 Closures

A **closure** is a function that **remembers the variables from its parent scope** even after the parent function has finished executing.

Real-World Analogy: Imagine you leave home for college. Even though you're no longer physically at home, you still **remember** your home WiFi password, your childhood recipes, and family traditions. You "closed over" those memories. A closure works the same way — a function "remembers" its parent's variables even after the parent is gone.

Basic Closure

```
function createGreeting(greeting) {  
  // 'greeting' lives in this scope  
  
  return function(name) {  
    // This inner function "closes over" 'greeting'  
    console.log(greeting + ", " + name + "!");  
  };  
}  
  
const sayHello = createGreeting("Hello");  
const sayHi = createGreeting("Hi");  
  
sayHello("Alice"); // "Hello, Alice!"  
sayHello("Bob");   // "Hello, Bob!"  
sayHi("Charlie");  // "Hi, Charlie!"
```

Step-by-step:

1. `createGreeting("Hello")` runs and creates `greeting = "Hello"`
2. It returns the inner function — this inner function **remembers** `greeting`
3. `createGreeting` finishes executing and is removed from the call stack
4. But `sayHello` still has access to `greeting = "Hello"` — that's the closure!

Practical Closure: Counter

```
function createCounter() {  
  let count = 0; // private variable  
  
  return {  
    increment: function() {  
      count++;  
      console.log("Count: " + count);  
    },  
    decrement: function() {  
      count--;  
      console.log("Count: " + count);  
    },  
  };  
}
```

```
    getCount: function() {
      return count;
    }
  };
}

const counter = createCounter();
counter.increment(); // Count: 1
counter.increment(); // Count: 2
counter.decrement(); // Count: 1
console.log(counter.getCount()); // 1
// console.log(count); // ❌ Error – count is private!
```

Why this matters: The variable `count` is **completely private**. No outside code can access or modify it directly — only through the methods provided. This is a real pattern used in professional code.

Advanced Closure: Loop Problem

```
// ❌ Common bug with var in loops
for (var i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);
  }, 1000);
}
// Output: 3, 3, 3 (not 0, 1, 2!)
```

Why? `var` is function-scoped. By the time `setTimeout` runs, the loop is finished and `i` is already `3`. All three functions share the **same** `i`.

Fix with let:

```
// ✅ Fix using let (block-scoped)
for (let i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);
  }, 1000);
}
// Output: 0, 1, 2
```

Fix with closure (old-school approach):

```
for (var i = 0; i < 3; i++) {
  (function(j) {
    setTimeout(function() {
      console.log(j);
    }, 1000);
  })(i);
}
```

```
}  
// Output: 0, 1, 2
```

 Exercise — Closures

Create a function called `bankAccount(initialBalance)` that returns an object with `deposit(amount)`, `withdraw(amount)`, and `balance()` methods. The balance should be private and only modifiable through the methods.

 Part 1 Summary

Concept	Key Point
JS Engine	Translates and runs your JavaScript code
Single-threaded	One task at a time
Call Stack	Tracks active functions (LIFO order)
Execution Context	Environment where code runs (GEC + FEC)
Hoisting	Declarations are moved to the top of scope during creation phase
Temporal Dead Zone	Period where let/const exist but can't be accessed
Scope	Where variables are visible (global, function, block)
Lexical Scope	Scope is determined by where code is written
Closures	Functions that remember their parent's variables after parent exits

Part 2 — The "this" Keyword and Function Methods

2.1 Understanding "this"

The `this` keyword in JavaScript refers to the **object that is currently executing the code**. Its value **changes depending on how and where a function is called**.

Real-World Analogy: Think of `this` like the word "I" in English. When **Alice** says "I am hungry," "I" refers to Alice. When **Bob** says "I am hungry," "I" refers to Bob. The word is the same, but the meaning depends on **who is speaking**. Similarly, `this` depends on **who is calling** the function.

2.2 "this" in Global Context

When used outside any function (in the global scope):

```
console.log(this); // In browser: Window object  
                  // In Node.js: {} (empty object in modules)
```

In a browser, `this` at the global level points to the `window` object — the big container that holds everything.

2.3 "this" in Objects

When a function is called as a **method of an object**, `this` refers to **that object**.

```
const person = {
  name: "Alice",
  age: 25,
  greet: function() {
    console.log("Hi, I'm " + this.name + " and I'm " + this.age);
  }
};

person.greet(); // "Hi, I'm Alice and I'm 25"
```

Step-by-step:

1. `person.greet()` is called — the function is called **on** the `person` object
 2. Inside `greet`, `this = person`
 3. So `this.name = "Alice"` and `this.age = 25`
-

2.4 "this" in Regular Functions

In a regular function (not a method), `this` depends on **strict mode**:

```
function showThis() {
  console.log(this);
}

showThis(); // In browser (non-strict): Window
            // In strict mode: undefined
```

Key Rule: When a function is called without an object (plain function call), `this` defaults to the global object (or `undefined` in strict mode).

The Tricky Part — Losing "this"

```
const car = {
  brand: "Toyota",
  getBrand: function() {
    console.log(this.brand);
  }
};
```

```
car.getBrand(); // "Toyota" ✓  
  
const detached = car.getBrand;  
detached(); // undefined ✗ - 'this' is now the global object!
```

Why? When you assign `car.getBrand` to `detached`, you're copying the function **without** the object. When you call `detached()`, there's no object before the dot, so `this` is not `car` anymore.

2.5 "this" in Arrow Functions

Arrow functions **do NOT have their own this**. They inherit `this` from their **surrounding scope** (lexical `this`).

```
const team = {  
  name: "Avengers",  
  members: ["Iron Man", "Thor", "Hulk"],  
  
  listMembers: function() {  
    // 'this' here is 'team'  
    this.members.forEach((member) => {  
      // Arrow function: 'this' is inherited from listMembers  
      console.log(member + " is in " + this.name);  
    });  
  }  
};  
  
team.listMembers();  
// "Iron Man is in Avengers"  
// "Thor is in Avengers"  
// "Hulk is in Avengers"
```

If we used a regular function instead:

```
const team2 = {  
  name: "Avengers",  
  members: ["Iron Man", "Thor"],  
  
  listMembers: function() {  
    this.members.forEach(function(member) {  
      // Regular function: 'this' is NOT team2!  
      console.log(member + " is in " + this.name); // ✗ undefined  
    });  
  }  
};
```

Rule of Thumb: Use arrow functions when you want to **keep** the parent's `this`. Use regular functions when you want `this` to be determined by **how** the function is called.

2.6 call()

The `call()` method lets you **call a function and explicitly set** what `this` should be.

```
function introduce(hobby, food) {  
  console.log(`I'm ${this.name}, I like ${hobby} and ${food}`);  
}  
  
const person1 = { name: "Alice" };  
const person2 = { name: "Bob" };  
  
introduce.call(person1, "reading", "pizza");  
// "I'm Alice, I like reading and pizza"  
  
introduce.call(person2, "gaming", "sushi");  
// "I'm Bob, I like gaming and sushi"
```

Syntax: `functionName.call(thisValue, arg1, arg2, ...)`

Step-by-step:

1. `introduce.call(person1, "reading", "pizza")` — call `introduce` with `this = person1`
2. Inside the function, `this.name = "Alice"`
3. Arguments are passed **individually** after the `this` value

2.7 apply()

`apply()` works exactly like `call()`, but you pass arguments as an **array**.

```
function introduce(hobby, food) {  
  console.log(`I'm ${this.name}, I like ${hobby} and ${food}`);  
}  
  
const person = { name: "Charlie" };  
  
introduce.apply(person, ["swimming", "tacos"]);  
// "I'm Charlie, I like swimming and tacos"
```

Syntax: `functionName.apply(thisValue, [arg1, arg2, ...])`

Memory Trick: Apply takes an **A**rray. Call takes **C**ommas.

Practical Use: Finding Max in an Array

```
const numbers = [5, 2, 9, 1, 7];

const max = Math.max.apply(null, numbers);
console.log(max); // 9

// Modern equivalent using spread:
const max2 = Math.max(...numbers);
console.log(max2); // 9
```

2.8 bind()

`bind()` creates a **new function** with `this` permanently set to a specific value. Unlike `call` and `apply`, it **does not call the function immediately**.

```
function greet() {
  console.log("Hello, I'm " + this.name);
}

const user = { name: "Diana" };

const greetDiana = greet.bind(user);

greetDiana(); // "Hello, I'm Diana"
greetDiana(); // "Hello, I'm Diana" (always Diana!)
```

Real-World Analogy: `call` is like asking someone "Hey, pretend to be Alice and say this." `bind` is like giving someone a permanent name tag that says "Alice" — every time they speak, they'll introduce themselves as Alice.

Practical Use: Event Handlers

```
const button = {
  label: "Submit",
  handleClick: function() {
    console.log("Clicked: " + this.label);
  }
};

// Without bind - 'this' would be the DOM element, not 'button'
// document.querySelector("btn").addEventListener("click",
// button.handleClick.bind(button));

// With bind - 'this' is permanently 'button'
const boundHandler = button.handleClick.bind(button);
boundHandler(); // "Clicked: Submit"
```

Partial Application with bind()

`bind()` can also pre-fill arguments:

```
function multiply(a, b) {
  return a * b;
}

const double = multiply.bind(null, 2); // 'a' is always 2
console.log(double(5)); // 10
console.log(double(10)); // 20

const triple = multiply.bind(null, 3);
console.log(triple(5)); // 15
```

2.9 Comparison Table

Method	Calls function?	Arguments	Returns
<code>call()</code>	☒ Yes	Comma-separated	Function's result
<code>apply()</code>	☒ Yes	Array	Function's result
<code>bind()</code>	☒ No	Comma-separated	New function

Exercise — "this" Keyword

```
const restaurant = {
  name: "Pizza Palace",
  menu: ["Margherita", "Pepperoni", "Hawaiian"],
  showMenu: function() {
    // Use forEach with an arrow function to print:
    // "Pizza Palace serves Margherita"
    // "Pizza Palace serves Pepperoni"
    // "Pizza Palace serves Hawaiian"
    // YOUR CODE HERE
  }
};
restaurant.showMenu();
```

Part 2 Summary

Concept	Key Point
<code>this</code> in global	<code>window</code> (browser) or <code>global</code> (Node.js)
<code>this</code> in methods	The object that owns the method

Concept	Key Point
this in functions	window (non-strict) or undefined (strict)
this in arrows	Inherited from parent scope (lexical)
call()	Invoke function with specific this + individual args
apply()	Invoke function with specific this + args as array
bind()	Returns NEW function with this permanently bound

Part 3 — Asynchronous JavaScript (Core Topic)

3.1 Synchronous vs Asynchronous JavaScript

Synchronous (Default Behavior)

Code runs **line by line**, each line waits for the previous one to finish.

```
console.log("Step 1: Order placed");
console.log("Step 2: Preparing food");
console.log("Step 3: Food ready!");
```

Output:

```
Step 1: Order placed
Step 2: Preparing food
Step 3: Food ready!
```

Asynchronous

Some tasks take time (loading data, reading files, waiting for user input). Instead of blocking everything, JavaScript can **start** a task and **move on**, then come back when the task is done.

```
console.log("Step 1: Order placed");

setTimeout(function() {
  console.log("Step 2: Food ready! (took 3 seconds)");
}, 3000);

console.log("Step 3: Meanwhile, setting the table");
```

Output:

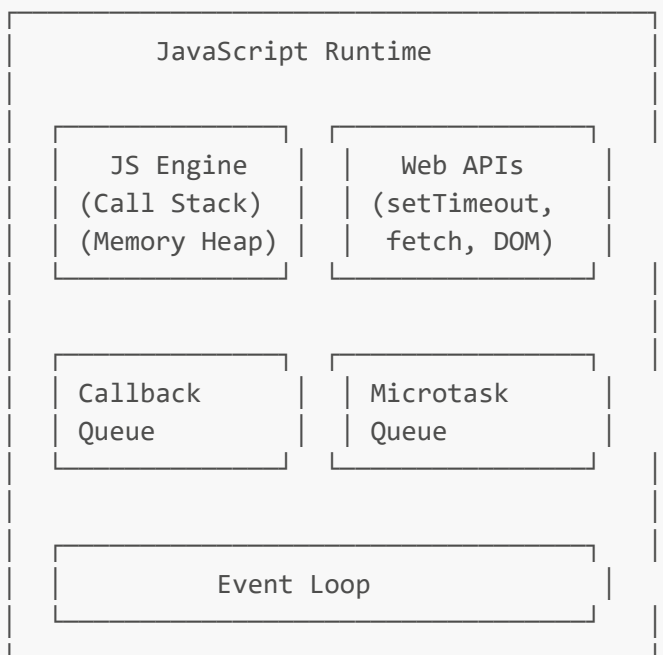
```
Step 1: Order placed
Step 3: Meanwhile, setting the table
Step 2: Food ready! (took 3 seconds) ← appears after 3 seconds
```

Real-World Analogy: Synchronous is like waiting at a coffee shop counter until your coffee is made.

Asynchronous is like ordering coffee, sitting down to read a book while it's being made, and the barista calls your name when it's ready.

3.2 JavaScript Runtime Environment

JavaScript doesn't work alone. It runs inside a **runtime environment** that provides extra tools:



- **JS Engine:** Runs your code (Call Stack + Memory Heap)
- **Web APIs:** Browser-provided features (setTimeout, fetch, DOM events)
- **Callback Queue (Task Queue):** Holds callbacks waiting to run
- **Microtask Queue:** Holds Promise callbacks (higher priority than callback queue)
- **Event Loop:** Checks if the Call Stack is empty, then moves tasks from queues to the stack

3.3 The Event Loop

The **Event Loop** is the mechanism that makes asynchronous JavaScript possible, despite being single-threaded.

Real-World Analogy: The Event Loop is like a waiter in a restaurant. The waiter (event loop) checks: "Is the chef (call stack) free? If yes, give the chef the next order from the queue. If not, wait."

How it works:

1. JavaScript runs code on the **Call Stack**
2. When it encounters an async operation (like `setTimeout`), it sends it to the **Web API**
3. When the Web API finishes, it puts the callback in the **Callback Queue**
4. The **Event Loop** constantly checks: "Is the Call Stack empty?"
5. If yes → move the next callback from the queue to the stack

```
console.log("1 - Start");

setTimeout(function() {
  console.log("2 - Timeout callback");
}, 0);

Promise.resolve().then(function() {
  console.log("3 - Promise callback");
});

console.log("4 - End");
```

Output:

```
1 - Start
4 - End
3 - Promise callback
2 - Timeout callback
```

Why this order?

1. "1 - Start" — synchronous, runs immediately
2. `setTimeout` — sent to Web API, callback goes to **Callback Queue**
3. `Promise.then` — callback goes to **Microtask Queue** (higher priority!)
4. "4 - End" — synchronous, runs immediately
5. Call Stack is empty → Event Loop checks Microtask Queue first → "3 - Promise callback"
6. Microtask Queue empty → Event Loop checks Callback Queue → "2 - Timeout callback"

Key Rule: Microtasks (Promises) always run before Macrotasks (`setTimeout`, `setInterval`).

3.4 Callback Functions

A **callback** is a function passed as an argument to another function, to be executed later.

```
function fetchData(callback) {
  console.log("Fetching data...");

  setTimeout(function() {
    const data = { name: "Alice", age: 25 };
    callback(data); // call the callback with the data
  }, 1000);
}
```

```
    }, 2000);  
  }  
  
  function displayData(data) {  
    console.log("Got data:", data);  
  }  
  
  fetchData(displayData);  
  // "Fetching data..."  
  // (after 2 seconds) "Got data: { name: 'Alice', age: 25 }"
```

3.5 Callback Hell

When you nest many callbacks inside each other, the code becomes deeply indented and hard to read. This is called **Callback Hell** (or the **Pyramid of Doom**).

```
// ✗ Callback Hell  
getUser(1, function(user) {  
  getOrders(user.id, function(orders) {  
    getOrderDetails(orders[0].id, function(details) {  
      getShippingInfo(details.shippingId, function(shipping) {  
        console.log("Shipping to:", shipping.address);  
        // ...and it keeps going deeper and deeper  
      });  
    });  
  });  
});
```

Problems with Callback Hell:

- Hard to read
- Hard to debug
- Hard to handle errors
- Hard to maintain

Solution: Use **Promises** or **async/await**.

3.6 Promises

A **Promise** is an object that represents the **eventual completion (or failure)** of an asynchronous operation.

Real-World Analogy: When you order food online, you get an order confirmation — that's a **promise**. The restaurant promises to either **deliver your food** (fulfilled/resolved) or **cancel your order** (rejected). While waiting, your order is **pending**.

Promise States

A Promise has exactly **three states**:

State	Description
<code>pending</code>	Initial state — neither fulfilled nor rejected
<code>fulfilled</code>	The operation completed successfully
<code>rejected</code>	The operation failed

Creating a Promise

```
const myPromise = new Promise(function(resolve, reject) {  
  const success = true;  
  
  if (success) {  
    resolve("Operation was successful! ✅");  
  } else {  
    reject("Something went wrong! ❌");  
  }  
});
```

resolve and reject

- `resolve(value)` — Call this when the operation **succeeds**. The promise becomes **fulfilled**.
- `reject(reason)` — Call this when the operation **fails**. The promise becomes **rejected**.

then(), catch(), and finally()

```
const orderFood = new Promise(function(resolve, reject) {  
  const foodAvailable = true;  
  
  setTimeout(function() {  
    if (foodAvailable) {  
      resolve("🍕 Pizza is ready!");  
    } else {  
      reject("Sorry, we're out of ingredients");  
    }  
  }, 2000);  
});  
  
orderFood  
  .then(function(result) {  
    console.log(result);           // "🍕 Pizza is ready!"  
    return "Now eating the pizza";  
  })  
  .then(function(message) {  
    console.log(message);         // "Now eating the pizza"  
  })  
  .catch(function(error) {
```

```
    console.log("Error:", error); // Runs if promise is rejected
  })
  .finally(function() {
    console.log("Order process complete"); // Always runs
  });
```

Explanation:

- `.then(callback)` — runs when promise is **resolved**. Receives the resolved value.
- `.catch(callback)` — runs when promise is **rejected**. Receives the error.
- `.finally(callback)` — runs **no matter what** (resolve or reject). Great for cleanup.

3.7 Promise Chaining

You can chain `.then()` calls to perform steps in sequence, where each step can be async.

```
function step1() {
  return new Promise(function(resolve) {
    setTimeout(function() {
      console.log("Step 1 complete");
      resolve(10);
    }, 1000);
  });
}

function step2(value) {
  return new Promise(function(resolve) {
    setTimeout(function() {
      console.log("Step 2 complete, received:", value);
      resolve(value * 2);
    }, 1000);
  });
}

function step3(value) {
  return new Promise(function(resolve) {
    setTimeout(function() {
      console.log("Step 3 complete, received:", value);
      resolve(value * 3);
    }, 1000);
  });
}

step1()
  .then(step2)
  .then(step3)
  .then(function(finalValue) {
    console.log("Final value:", finalValue);
  })
  .catch(function(error) {
```

```
    console.log("Error:", error);
  });

// Output (each 1 second apart):
// Step 1 complete
// Step 2 complete, received: 10
// Step 3 complete, received: 20
// Final value: 60
```

3.8 Promise Helper Methods

JavaScript provides several static methods on the `Promise` object to handle **multiple promises** at once:

Promise.all()

Waits for **ALL** promises to resolve. Rejects if **any one** rejects.

```
const p1 = fetch("https://api.example.com/users");
const p2 = fetch("https://api.example.com/posts");
const p3 = fetch("https://api.example.com/comments");

Promise.all([p1, p2, p3])
  .then(function(results) {
    console.log("All 3 requests completed!", results);
  })
  .catch(function(error) {
    console.log("At least one failed:", error);
  });
```

Analogy: You're waiting for 3 friends to arrive at a party. `Promise.all` means the party starts only when **everyone** arrives. If one friend cancels, the party is cancelled.

Promise.allSettled()

Waits for **ALL** promises to finish (whether resolved or rejected). Never rejects.

```
const p1 = Promise.resolve("Success 1");
const p2 = Promise.reject("Error 2");
const p3 = Promise.resolve("Success 3");

Promise.allSettled([p1, p2, p3])
  .then(function(results) {
    results.forEach(function(result) {
      console.log(result.status, result.value || result.reason);
    });
  });

// Output:
```

```
// "fulfilled" "Success 1"
// "rejected" "Error 2"
// "fulfilled" "Success 3"
```

Promise.race()

Returns the result of the **first** promise to settle (resolve or reject).

```
const slow = new Promise(function(resolve) {
  setTimeout(function() { resolve("Slow!"); }, 3000);
});

const fast = new Promise(function(resolve) {
  setTimeout(function() { resolve("Fast!"); }, 1000);
});

Promise.race([slow, fast])
  .then(function(result) {
    console.log(result); // "Fast!" – whichever finishes first
  });
```

Promise.any()

Returns the result of the **first promise to resolve** (ignores rejections unless ALL reject).

```
const p1 = Promise.reject("Error 1");
const p2 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Success from p2"); }, 2000);
});
const p3 = new Promise(function(resolve) {
  setTimeout(function() { resolve("Success from p3"); }, 1000);
});

Promise.any([p1, p2, p3])
  .then(function(result) {
    console.log(result); // "Success from p3" – first to RESOLVE
  });
```

Method	Waits for	Rejects when
<code>Promise.all()</code>	All resolved	Any one rejects
<code>Promise.allSettled()</code>	All settled	Never
<code>Promise.race()</code>	First settled	First settled rejects
<code>Promise.any()</code>	First resolved	All reject

3.9 Async / Await

`async/await` is **syntactic sugar on top of Promises**. It makes asynchronous code look and feel like synchronous code.

```
// Using Promises:
function getUser() {
  return fetch("https://jsonplaceholder.typicode.com/users/1")
    .then(function(response) {
      return response.json();
    })
    .then(function(user) {
      console.log(user.name);
    });
}

// Using async/await (same thing, cleaner):
async function getUser() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users/1");
  const user = await response.json();
  console.log(user.name);
}

getUser();
```

Key Rules:

- `async` before a function makes it return a Promise
- `await` pauses the function until the Promise resolves
- You can only use `await` inside an `async` function

Real-World Analogy: Without `async/await`, you're sending messages back and forth: "When you're done, call me back." With `async/await`, you're just saying: "I'll wait right here until you're done, then continue."

3.10 Error Handling in Async Code

Using `try/catch` with `async/await`

```
async function fetchUserData() {
  try {
    const response = await fetch("https://invalid-url.example.com");

    if (!response.ok) {
      throw new Error("HTTP error! Status: " + response.status);
    }

    const data = await response.json();
    console.log("User data:", data);
  } catch (error) {
```

```
        console.log("Failed to fetch:", error.message);
    } finally {
        console.log("Fetch attempt complete");
    }
}

fetchUserData();
```

Using .catch() with Promises

```
fetch("https://invalid-url.example.com")
  .then(function(response) {
    if (!response.ok) {
      throw new Error("HTTP error!");
    }
    return response.json();
  })
  .then(function(data) {
    console.log(data);
  })
  .catch(function(error) {
    console.log("Error:", error.message);
  });
```

 Exercise — Async/Await

Write an `async` function called `getMultipleUsers` that fetches 3 users from `https://jsonplaceholder.typicode.com/users/1`, `/users/2`, and `/users/3` simultaneously using `Promise.all()`, then logs all their names.

 Part 3 Summary

Concept	Key Point
Synchronous	Code runs line by line, each line waits
Asynchronous	Start a task, move on, come back when done
Event Loop	Moves tasks from queue to call stack when stack is empty
Callbacks	Functions passed as arguments, called later
Callback Hell	Deeply nested callbacks — hard to read and maintain
Promises	Objects representing future results (pending/fulfilled/rejected)
<code>.then()</code>	Runs on successful resolution
<code>.catch()</code>	Runs on rejection

Concept	Key Point
<code>.finally()</code>	Runs always, regardless of outcome
<code>Promise.all()</code>	Waits for all to resolve
<code>Promise.allSettled()</code>	Waits for all to settle
<code>Promise.race()</code>	Returns first to settle
<code>Promise.any()</code>	Returns first to resolve
<code>async/await</code>	Cleaner syntax for Promises

Part 4 — Error Handling

4.1 try

The `try` block lets you **test** a block of code for errors. If an error occurs inside `try`, JavaScript doesn't crash — instead, control jumps to the `catch` block.

Real-World Analogy: `try` is like putting on a seatbelt before driving. You're preparing for the possibility that something might go wrong.

```
try {
  let result = 10 / 0;
  console.log(result); // Infinity (not an error in JS)

  let data = JSON.parse("invalid json");
  console.log(data); // ✗ This will throw an error
}
// The catch block handles the error
```

4.2 catch

The `catch` block runs **only if an error occurs** inside the `try` block. It receives the error object.

```
try {
  let data = JSON.parse("this is not valid JSON");
} catch (error) {
  console.log("Error name:", error.name);           // SyntaxError
  console.log("Error message:", error.message);    // Unexpected token...
}

console.log("Program continues running!"); // ☑ Code doesn't crash
```

Step-by-step:

1. `JSON.parse("this is not valid JSON")` — this throws a `SyntaxError`
 2. JavaScript immediately jumps to the `catch` block
 3. The `error` parameter contains information about what went wrong
 4. After `catch`, the rest of the program continues normally
-

4.3 finally

The `finally` block runs **no matter what** — whether there was an error or not. It's perfect for **cleanup** tasks.

```
function readFile(filename) {
  console.log("Opening file:", filename);

  try {
    if (filename === "missing.txt") {
      throw new Error("File not found!");
    }
    console.log("Reading file contents...");
    return "file data";
  } catch (error) {
    console.log("Error:", error.message);
    return null;
  } finally {
    console.log("Closing file connection"); // ALWAYS runs
  }
}

readFile("data.txt");
// Opening file: data.txt
// Reading file contents...
// Closing file connection

readFile("missing.txt");
// Opening file: missing.txt
// Error: File not found!
// Closing file connection
```

4.4 throw

The `throw` statement lets you **create your own errors**. You can throw any value, but it's best to throw `Error` objects.

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero!");
  }
  return a / b;
}
```

```
try {
  console.log(divide(10, 2)); // 5
  console.log(divide(10, 0)); // ✖ Throws error
} catch (error) {
  console.log("Caught:", error.message); // "Caught: Cannot divide by zero!"
}
```

Throwing Different Error Types

```
function validateAge(age) {
  if (typeof age !== "number") {
    throw new TypeError("Age must be a number");
  }
  if (age < 0 || age > 150) {
    throw new RangeError("Age must be between 0 and 150");
  }
  return "Valid age: " + age;
}

try {
  console.log(validateAge(25)); // "Valid age: 25"
  console.log(validateAge("hello")); // ✖ TypeError
} catch (error) {
  console.log(error.name + ": " + error.message);
}
```

Built-in Error Types:

Error Type	When to use
Error	General errors
TypeError	Wrong data type
RangeError	Number out of range
ReferenceError	Variable doesn't exist
SyntaxError	Code has invalid syntax

4.5 Custom Errors

You can create your own error classes for specific situations in your application.

```
class ValidationError extends Error {
  constructor(field, message) {
    super(message); // Call parent constructor
    this.name = "ValidationError";
  }
}
```

```
        this.field = field;
    }
}

class AuthenticationError extends Error {
    constructor(message) {
        super(message);
        this.name = "AuthenticationError";
        this.statusCode = 401;
    }
}

function loginUser(username, password) {
    if (!username) {
        throw new ValidationError("username", "Username is required");
    }
    if (password.length < 8) {
        throw new ValidationError("password", "Password must be at least 8
characters");
    }
    if (username !== "admin" || password !== "password123") {
        throw new AuthenticationError("Invalid credentials");
    }
    return "Login successful!";
}

try {
    loginUser("admin", "short");
} catch (error) {
    if (error instanceof ValidationError) {
        console.log(`Validation failed on '${error.field}': ${error.message}`);
        // "Validation failed on 'password': Password must be at least 8 characters"
    } else if (error instanceof AuthenticationError) {
        console.log(`Auth error (${error.statusCode}): ${error.message}`);
    } else {
        console.log("Unknown error:", error);
    }
}
```

Exercise — Error Handling

Write a function `withdrawMoney(balance, amount)` that:

- Throws a `TypeError` if either argument is not a number
- Throws a `RangeError` if amount is negative
- Throws a custom `InsufficientFundsError` if amount > balance
- Returns the remaining balance if successful

Part 4 Summary

Concept	Key Point
<code>try</code>	Wraps code that might throw an error
<code>catch</code>	Handles the error if one is thrown
<code>finally</code>	Runs always, great for cleanup
<code>throw</code>	Creates and throws your own errors
Custom Errors	Extend <code>Error</code> class for specific error types

Part 5 — Advanced Function Concepts

5.1 Higher-Order Functions

A **Higher-Order Function** is a function that either:

- **Takes a function as an argument**, OR
- **Returns a function**

Real-World Analogy: A manager is a higher-order person — they don't do the task themselves, they **delegate** the task to others (pass functions) or **create teams** to do work later (return functions).

Functions that Accept Functions

```
// map, filter, and reduce are higher-order functions
const numbers = [1, 2, 3, 4, 5];

// map – transform each element
const doubled = numbers.map(function(num) {
  return num * 2;
});
console.log(doubled); // [2, 4, 6, 8, 10]

// filter – keep elements that pass a test
const evens = numbers.filter(function(num) {
  return num % 2 === 0;
});
console.log(evens); // [2, 4]

// reduce – combine all elements into a single value
const sum = numbers.reduce(function(accumulator, current) {
  return accumulator + current;
}, 0);
console.log(sum); // 15

// forEach – do something with each element
numbers.forEach(function(num) {
  console.log("Number:", num);
});
```

```
// find – find the first element that passes a test
const firstBig = numbers.find(function(num) {
  return num > 3;
});
console.log(firstBig); // 4

// some – check if at least one element passes
const hasNegative = numbers.some(function(num) {
  return num < 0;
});
console.log(hasNegative); // false

// every – check if ALL elements pass
const allPositive = numbers.every(function(num) {
  return num > 0;
});
console.log(allPositive); // true
```

Building Your Own Higher-Order Function

```
function repeat(times, action) {
  for (let i = 0; i < times; i++) {
    action(i);
  }
}

repeat(3, function(i) {
  console.log("Attempt #" + (i + 1));
});
// Attempt #1
// Attempt #2
// Attempt #3
```

5.2 Closures in Depth

We covered closures in Part 1. Now let's see **advanced real-world uses**.

Use Case 1: Function Factories

```
function createMultiplier(multiplier) {
  return function(number) {
    return number * multiplier;
  };
}

const double = createMultiplier(2);
const triple = createMultiplier(3);
```

```
const tenTimes = createMultiplier(10);

console.log(double(5)); // 10
console.log(triple(5)); // 15
console.log(tenTimes(5)); // 50
```

Use Case 2: Private State (Data Privacy)

```
function createUser(name) {
  let loginCount = 0; // private – cannot be accessed directly

  return {
    getName: function() { return name; },
    login: function() {
      loginCount++;
      console.log(`${name} logged in (total: ${loginCount})`);
    },
    getLoginCount: function() { return loginCount; }
  };
}

const user = createUser("Alice");
user.login(); // Alice logged in (total: 1)
user.login(); // Alice logged in (total: 2)
console.log(user.getLoginCount()); // 2
// console.log(loginCount); // ✗ Error – private!
```

Use Case 3: Rate Limiter

```
function createRateLimiter(maxCalls, timeWindow) {
  let callCount = 0;

  setInterval(function() {
    callCount = 0; // Reset every timeWindow ms
  }, timeWindow);

  return function(action) {
    if (callCount < maxCalls) {
      callCount++;
      action();
    } else {
      console.log("Rate limit exceeded. Try again later.");
    }
  };
}

const limiter = createRateLimiter(3, 10000); // 3 calls per 10 seconds
```

5.3 Currying

Currying transforms a function that takes multiple arguments into a sequence of functions, each taking **one argument**.

Real-World Analogy: Ordering a custom sandwich. Instead of saying everything at once ("turkey, wheat, lettuce, mustard"), you go through stations: First you pick bread → then protein → then toppings → then sauce. Each step takes one choice.

```
// Normal function
function add(a, b, c) {
  return a + b + c;
}
console.log(add(1, 2, 3)); // 6

// Curried version
function curriedAdd(a) {
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}
console.log(curriedAdd(1)(2)(3)); // 6

// Arrow function version (cleaner)
const curriedAddArrow = (a) => (b) => (c) => a + b + c;
console.log(curriedAddArrow(1)(2)(3)); // 6
```

Practical Currying: Configurable Logger

```
function createLogger(level) {
  return function(module) {
    return function(message) {
      console.log(`[${level}] [${module}] ${message}`);
    };
  };
}

const errorLog = createLogger("ERROR");
const authError = errorLog("AUTH");
const dbError = errorLog("DATABASE");

authError("Invalid password"); // [ERROR] [AUTH] Invalid password
authError("Token expired"); // [ERROR] [AUTH] Token expired
dbError("Connection timeout"); // [ERROR] [DATABASE] Connection timeout
```

Generic Curry Helper

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function(...moreArgs) {
      return curried.apply(this, args.concat(moreArgs));
    };
  };
}

function calculatePrice(tax, discount, price) {
  return price * (1 + tax) * (1 - discount);
}

const curriedPrice = curry(calculatePrice);
const withTax = curriedPrice(0.1);           // 10% tax
const withTaxAndDiscount = withTax(0.2);     // 20% discount
console.log(withTaxAndDiscount(100));        // 88
```

5.4 Function Composition

Function Composition is combining two or more functions to produce a new function, where each function's output becomes the next function's input.

Real-World Analogy: An assembly line in a factory. Raw material → Station 1 (cut) → Station 2 (paint) → Station 3 (package) → Final product. Each station takes the previous output and produces new output.

```
// Simple functions
const addTen = (x) => x + 10;
const double = (x) => x * 2;
const square = (x) => x * x;

// Manual composition
const result = square(double(addTen(5)));
console.log(result); // square(double(15)) = square(30) = 900

// Compose utility (right to left)
function compose(...functions) {
  return function(value) {
    return functions.reduceRight(function(acc, fn) {
      return fn(acc);
    }, value);
  };
}

const transform = compose(square, double, addTen);
console.log(transform(5)); // 900
```

```
// Pipe utility (left to right – more intuitive)
function pipe(...functions) {
  return function(value) {
    return functions.reduce(function(acc, fn) {
      return fn(acc);
    }, value);
  };
}

const process = pipe(addTen, double, square);
console.log(process(5)); // 5 → 15 → 30 → 900
```

5.5 Debouncing

Debouncing ensures a function is only called after a certain amount of time has passed **since the last call**. If the function is called again before the delay, the timer resets.

Real-World Analogy: An elevator door. Every time someone approaches, the door reopens and the timer resets. The door only closes when no one has approached for a few seconds.

```
function debounce(fn, delay) {
  let timeoutId;

  return function(...args) {
    clearTimeout(timeoutId); // Cancel previous timer

    timeoutId = setTimeout(function() {
      fn.apply(this, args); // Call the function after delay
    }, delay);
  };
}

// Usage: Search input
const searchAPI = function(query) {
  console.log("Searching for:", query);
};

const debouncedSearch = debounce(searchAPI, 500);

// Simulating rapid typing:
debouncedSearch("h"); // timer starts
debouncedSearch("he"); // timer resets
debouncedSearch("hel"); // timer resets
debouncedSearch("hell"); // timer resets
debouncedSearch("hello"); // timer resets
// After 500ms of no typing: "Searching for: hello" (only ONE call!)
```

Use cases: Search boxes, window resize handlers, form validation.

5.6 Throttling

Throttling ensures a function is called at most **once every specified time period**, regardless of how many times it's triggered.

Real-World Analogy: A speed limit on a highway. No matter how fast you want to drive, you can only go 60 mph. The road "throttles" your speed.

```
function throttle(fn, limit) {
  let lastCall = 0;

  return function(...args) {
    const now = Date.now();

    if (now - lastCall >= limit) {
      lastCall = now;
      fn.apply(this, args);
    }
  };
}

// Usage: Scroll event handler
const handleScroll = function() {
  console.log("Scroll position:", window.scrollY);
};

const throttledScroll = throttle(handleScroll, 200);
// Even if scroll fires 100 times/second, handleScroll runs at most every 200ms
```

Debounce vs Throttle

Feature	Debounce	Throttle
When it runs	After a pause in calls	At regular intervals
Use case	Search input, form validation	Scroll events, game loops
Behavior	Waits for quiet period	Limits execution rate

5.7 Memoization

Memoization is caching the results of expensive function calls so you don't compute the same thing twice.

Real-World Analogy: A student doing homework. The first time they solve a problem, it takes effort. But they write the answer in a notebook. Next time the same problem appears, they just look at their notes instead of solving it from scratch.

```
function memoize(fn) {
  const cache = {};
```

```
return function(...args) {
  const key = JSON.stringify(args);

  if (cache[key] !== undefined) {
    console.log("📦 From cache:", key);
    return cache[key];
  }

  console.log("🔗 Computing:", key);
  const result = fn.apply(this, args);
  cache[key] = result;
  return result;
};

// Expensive function: Fibonacci
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

const memoizedFib = memoize(function(n) {
  if (n <= 1) return n;
  return memoizedFib(n - 1) + memoizedFib(n - 2);
});

console.log(memoizedFib(40)); // Fast! Uses cache for repeated calls
```

Practical Memoization: API Caching

```
const memoizedFetch = memoize(async function(url) {
  const response = await fetch(url);
  return response.json();
});

// First call – hits the network
await memoizedFetch("https://api.example.com/users/1");

// Second call – returns cached result instantly
await memoizedFetch("https://api.example.com/users/1");
```

🔧 Exercise — Advanced Functions

1. Create a **pipe** function that takes any number of functions and returns a new function that runs them left-to-right.
2. Create a **debounce** function and test it with `console.log`.
3. Create a memoized version of a function that checks if a number is prime.

 Part 5 Summary

Concept	Key Point
Higher-Order Functions	Functions that take or return other functions
Closures (advanced)	Private state, factories, rate limiters
Currying	Transform multi-arg function into chain of single-arg ones
Function Composition	Chain functions: output of one is input of next
Debouncing	Call function only after a pause from the last trigger
Throttling	Call function at most once per time interval
Memoization	Cache results to avoid repeated computation

Part 6 — Special JavaScript Patterns

6.1 IIFE (Immediately Invoked Function Expression)

An IIFE is a function that **runs immediately** as soon as it's defined.

Real-World Analogy: Think of a self-destructing note in a spy movie. You write the message, it's read immediately, and then it's gone. An IIFE creates a scope, runs the code, and doesn't pollute the global namespace.

Syntax

```
// Basic IIFE
(function() {
  console.log("I run immediately!");
})();

// With arrow function
(() => {
  console.log("Arrow IIFE!");
})();

// With parameters
(function(name) {
  console.log("Hello, " + name);
})("Alice");

// Returning a value
const result = (function() {
  const secret = 42;
  return secret * 2;
})();
console.log(result); // 84
```

Why Use IIFE?

1. Avoid polluting global scope:

```
// Without IIFE – variables leak to global scope
var counter = 0; // ✗ Global variable

// With IIFE – variables stay private
(function() {
  var counter = 0; // ✔ Private to this scope
  counter++;
  console.log(counter); // 1
})();
// console.log(counter); // ✗ ReferenceError (if no global var)
```

2. Create isolated modules (pre-ES6):

```
const myApp = (function() {
  let privateData = "secret";

  function privateMethod() {
    return privateData.toUpperCase();
  }

  return {
    getData: function() {
      return privateMethod();
    }
  };
})();

console.log(myApp.getData()); // "SECRET"
// console.log(myApp.privateData); // undefined – truly private
```

6.2 Module Pattern

The Module Pattern uses closures and IIFEs to create **self-contained modules** with private and public parts.

Real-World Analogy: A vending machine. You can see the buttons (public interface) and use them to get snacks. But you can't access the internal mechanism, inventory count, or cash box (private data).

```
const ShoppingCart = (function() {
  // Private variables
  let items = [];
  let totalPrice = 0;
```

```

// Private function
function calculateTotal() {
  totalPrice = items.reduce(function(sum, item) {
    return sum + item.price * item.quantity;
  }, 0);
}

// Public API (returned object)
return {
  addItem: function(name, price, quantity) {
    items.push({ name, price, quantity: quantity || 1 });
    calculateTotal();
    console.log(`Added ${name} to cart`);
  },

  removeItem: function(name) {
    items = items.filter(function(item) {
      return item.name !== name;
    });
    calculateTotal();
    console.log(`Removed ${name} from cart`);
  },

  getTotal: function() {
    return "$" + totalPrice.toFixed(2);
  },

  getItems: function() {
    return [...items]; // Return a copy, not the actual array
  },

  clear: function() {
    items = [];
    totalPrice = 0;
    console.log("Cart cleared");
  }
};
})();

ShoppingCart.addItem("Laptop", 999.99); // Added Laptop to cart
ShoppingCart.addItem("Mouse", 29.99, 2); // Added Mouse to cart
console.log(ShoppingCart.getTotal()); // "$1059.97"
console.log(ShoppingCart.getItems()); // [{...}, {...}]
// console.log(ShoppingCart.items); // undefined – private!

```

6.3 Factory Functions

A **Factory Function** is a function that creates and returns objects. It's an alternative to using `class` or `new`.

Real-World Analogy: A car factory. You tell the factory what type of car you want (color, model, engine), and it builds one for you. You don't need to know how the factory works internally.

```
function createPlayer(name, level, health) {
  // Private variable
  let xp = 0;

  return {
    name,
    level,
    health,

    attack: function(target) {
      const damage = level * 10;
      console.log(`${name} attacks ${target.name} for ${damage} damage!`);
      target.health -= damage;
      xp += 15;
    },

    heal: function(amount) {
      health += amount;
      console.log(`${name} heals for ${amount}. Health: ${health}`);
    },

    getXP: function() {
      return xp;
    },

    getInfo: function() {
      return `${name} | Level: ${level} | HP: ${health} | XP: ${xp}`;
    }
  };
}

const warrior = createPlayer("Aragorn", 5, 100);
const mage = createPlayer("Gandalf", 8, 80);

warrior.attack(mage);
console.log(mage.getInfo()); // "Gandalf | Level: 8 | HP: 30 | XP: 0"
console.log(warrior.getXP()); // 15
```

Factory with Composition (Mixins)

```
// Behavior mixins
const canSwim = (state) => ({
  swim: () => console.log(`${state.name} is swimming`)
});

const canFly = (state) => ({
  fly: () => console.log(`${state.name} is flying`)
});

const canWalk = (state) => ({
```

```
    walk: () => console.log(`${state.name} is walking`)
  });

  // Factory using composition
  function createDuck(name) {
    const state = { name };

    return Object.assign(
      {},
      canSwim(state),
      canFly(state),
      canWalk(state),
      { getName: () => state.name }
    );
  }

  function createPenguin(name) {
    const state = { name };

    return Object.assign(
      {},
      canSwim(state),
      canWalk(state), // No canFly – penguins can't fly!
      { getName: () => state.name }
    );
  }

  const donald = createDuck("Donald");
  donald.swim(); // "Donald is swimming"
  donald.fly(); // "Donald is flying"

  const tux = createPenguin("Tux");
  tux.swim();    // "Tux is swimming"
  tux.walk();    // "Tux is walking"
  // tux.fly();  // ✗ TypeError – penguins can't fly!
```

Factory vs Class

Feature	Factory Function	Class
Syntax	Regular function	<code>class</code> keyword
<code>new</code> keyword	Not needed	Required
<code>this</code> issues	No <code>this</code> problems	<code>this</code> can be confusing
Private data	True privacy (closures)	Needs <code>#</code> prefix (newer)
Inheritance	Composition (flexible)	Prototype chain (rigid)

 Exercise — Patterns

1. Create an IIFE that generates a unique ID each time a function is called (using a closure inside the IIFE).

- 2. Create a module pattern for a `TodoList` with `addTodo`, `removeTodo`, `getTodos`, and `getCount` methods.
- 3. Create a factory function `createBankAccount` with `deposit`, `withdraw`, and `getBalance` methods.

 Part 6 Summary

Concept	Key Point
IIFE	Runs immediately, creates private scope
Module Pattern	Uses IIFE + closures for private/public separation
Factory Functions	Functions that create and return objects (no <code>new</code>)
Composition	Combine behaviors by mixing objects (more flexible)

Part 7 — Prototypes and Object Internals

7.1 What is a Prototype?

Every JavaScript object has a hidden internal property called `[[Prototype]]` (accessible via `__proto__` or `Object.getPrototypeOf()`). This is a reference to **another object** from which it can inherit properties and methods.

Real-World Analogy: A prototype is like a **parent**. If a child doesn't know something, they ask their parent. If the parent doesn't know, they ask the grandparent. This chain of asking continues until someone has the answer (or nobody does).

```
const animal = {
  isAlive: true,
  eat: function() {
    console.log(this.name + " is eating");
  }
};

const dog = Object.create(animal); // dog's prototype is animal
dog.name = "Buddy";
dog.bark = function() {
  console.log("Woof!");
};

dog.bark();           // "Woof!"           - own property
dog.eat();            // "Buddy is eating" - inherited from animal
console.log(dog.isAlive); // true          - inherited from animal
```

Step-by-step:

- 1. `Object.create(animal)` creates a new object (`dog`) whose prototype is `animal`

2. When we call `dog.eat()`, JavaScript looks for `eat` on `dog` — not found
3. JavaScript then checks `dog`'s prototype (`animal`) — found! Runs it.
4. `this.name` still refers to `dog` because `dog` called the method

7.2 Prototype Chain

The **Prototype Chain** is the series of links from one object to its prototype, to that prototype's prototype, and so on, until it reaches `null`.

```
dog → animal → Object.prototype → null
```

```
const vehicle = {
  hasEngine: true,
  start: function() {
    console.log("Starting " + this.type);
  }
};

const car = Object.create(vehicle);
car.type = "Car";
car.doors = 4;

const tesla = Object.create(car);
tesla.brand = "Tesla";
tesla.type = "Electric Car";

tesla.start();           // "Starting Electric Car"
console.log(tesla.doors); // 4 (from car)
console.log(tesla.hasEngine); // true (from vehicle)
console.log(tesla.brand);  // "Tesla" (own property)

// Visualizing the chain:
console.log(tesla.__proto__ === car);           // true
console.log(car.__proto__ === vehicle);         // true
console.log(vehicle.__proto__ === Object.prototype); // true
console.log(Object.prototype.__proto__ === null); // true
```

How Property Lookup Works

Looking for `tesla.doors`:

1. Check `tesla` → not found
2. Check `car` (`tesla`'s prototype) → ☒ found! Return 4

Looking for `tesla.toString()`:

1. Check `tesla` → not found
2. Check `car` → not found

3. Check vehicle → not found
4. Check Object.prototype → ☒ found! (built-in method)

Looking for tesla.fly():

1. Check tesla → not found
2. Check car → not found
3. Check vehicle → not found
4. Check Object.prototype → not found
5. Reached null → return undefined

7.3 Prototype Inheritance

Using Constructor Functions (Pre-ES6)

```
function Animal(name, sound) {
  this.name = name;
  this.sound = sound;
}

Animal.prototype.speak = function() {
  console.log(this.name + " says " + this.sound);
};

Animal.prototype.sleep = function() {
  console.log(this.name + " is sleeping 🐾");
};

function Dog(name, breed) {
  Animal.call(this, name, "Woof"); // Call parent constructor
  this.breed = breed;
}

// Set up inheritance
Dog.prototype = Object.create(Animal.prototype);
Dog.prototype.constructor = Dog; // Fix constructor reference

// Add Dog-specific methods
Dog.prototype.fetch = function(item) {
  console.log(this.name + " fetches the " + item + "! 🐾");
};

const rex = new Dog("Rex", "German Shepherd");
rex.speak(); // "Rex says Woof"
rex.sleep(); // "Rex is sleeping 🐾"
rex.fetch("ball"); // "Rex fetches the ball! 🐾"

console.log(rex instanceof Dog); // true
console.log(rex instanceof Animal); // true
```

Using ES6 Classes (Modern)

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  speak() {
    console.log(`${this.name} says ${this.sound}`);
  }

  sleep() {
    console.log(`${this.name} is sleeping 🐾`);
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name, "Woof"); // Call parent constructor
    this.breed = breed;
  }

  fetch(item) {
    console.log(`${this.name} fetches the ${item}! 🐾`);
  }
}

const rex = new Dog("Rex", "German Shepherd");
rex.speak(); // "Rex says Woof"
rex.fetch("ball"); // "Rex fetches the ball! 🐾"
```

7.4 Difference Between Prototypes and Classes

Important Truth: JavaScript classes are just **syntactic sugar** over prototypes. Under the hood, they work exactly the same way.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log("Hi, I'm " + this.name);
  }
}

// This is essentially the same as:
function PersonOld(name) {
  this.name = name;
```

```
}
PersonOld.prototype.greet = function() {
  console.log("Hi, I'm " + this.name);
};

// Proof:
console.log(typeof Person); // "function" – classes ARE functions!
```

Feature	Prototypes	Classes
Introduced	JavaScript's birth (1995)	ES6 (2015)
Syntax	Verbose, manual setup	Clean, familiar (like Java/C++)
Under the hood	Direct prototype manipulation	Same prototype manipulation!
<code>super</code> keyword	Use <code>Parent.call(this)</code>	Use <code>super()</code>
Private fields	Use closures	Use <code>#fieldName</code>
Static methods	Attach to constructor directly	Use <code>static</code> keyword
Hoisting	Function declarations are hoisted	Classes are NOT hoisted

When to Use Which

- Use **classes** for most new code — cleaner syntax, better readability
- Understand **prototypes** because classes use them under the hood, and you'll see them in older codebases
- Use **factory functions** when you need true privacy or flexible composition

 Exercise — Prototypes

1. Create a `Shape` class with a `describe()` method. Then create `Circle` and `Rectangle` that extend `Shape`.
2. Verify that `new Circle()` is an `instanceof` both `Circle` and `Shape`.
3. Add a `getArea()` method to each subclass.

 Part 7 Summary

Concept	Key Point
Prototype	A hidden object that another object inherits from
Prototype Chain	Chain of prototypes searched when a property isn't found
Prototype Inheritance	Objects inherit properties/methods through the chain
Classes vs Prototypes	Classes are syntactic sugar over prototypes

Part 8 — Modern JavaScript (ES6+)

8.1 let vs const vs var

```
// var – function-scoped, hoisted with undefined, can be redeclared
var x = 1;
var x = 2; // ☒ No error
console.log(x); // 2

// let – block-scoped, hoisted but in TDZ, cannot be redeclared
let y = 1;
// let y = 2; // ☒ SyntaxError: already declared
y = 2; // ☒ Can reassign

// const – block-scoped, hoisted but in TDZ, cannot reassign
const z = 1;
// z = 2; // ☒ TypeError: Assignment to constant variable

// BUT const objects/arrays can be mutated!
const person = { name: "Alice" };
person.name = "Bob"; // ☒ This works!
person.age = 25;      // ☒ Adding properties works too
// person = {};       // ☒ Cannot reassign the variable itself
```

Feature	var	let	const
Scope	Function	Block	Block
Hoisting	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Redeclare	☒ Yes	☒ No	☒ No
Reassign	☒ Yes	☒ Yes	☒ No

Best Practice: Use `const` by default. Use `let` only when you need to reassign. Avoid `var`.

8.2 Template Literals

Template literals use **backticks** (```) instead of quotes, and allow **embedded expressions** and **multi-line strings**.

```
const name = "Alice";
const age = 25;

// Old way
const greeting1 = "Hello, " + name + "! You are " + age + " years old.";

// Template literal (cleaner)
const greeting2 = `Hello, ${name}! You are ${age} years old.`;
```

```
// Expressions inside ${}
const price = 19.99;
const tax = 0.1;
console.log(`Total: ${$(price * (1 + tax)).toFixed(2)}`); // "Total: $21.99"

// Multi-line strings
const html = `
  <div class="card">
    <h2>${name}</h2>
    <p>Age: ${age}</p>
  </div>
`;

// Tagged template literals (advanced)
function highlight(strings, ...values) {
  return strings.reduce(function(result, str, i) {
    return result + str + (values[i] ? `**${values[i]}**` : "");
  }, "");
}

const item = "laptop";
const cost = 999;
const message = highlight`I bought a ${item} for ${cost}`;
console.log(message); // "I bought a **laptop** for $**999**"
```

8.3 Destructuring

Destructuring lets you **unpack values** from arrays or properties from objects into individual variables.

Object Destructuring

```
const user = {
  name: "Alice",
  age: 25,
  email: "alice@example.com",
  address: {
    city: "New York",
    country: "USA"
  }
};

// Old way
const name1 = user.name;
const age1 = user.age;

// Destructuring
const { name, age, email } = user;
console.log(name); // "Alice"
console.log(age); // 25
console.log(email); // "alice@example.com"
```

```
// With renaming
const { name: userName, age: userAge } = user;
console.log(userName); // "Alice"

// With default values
const { phone = "Not provided" } = user;
console.log(phone); // "Not provided"

// Nested destructuring
const { address: { city, country } } = user;
console.log(city); // "New York"

// In function parameters
function displayUser({ name, age, email }) {
  console.log(`${name} (${age}) - ${email}`);
}
displayUser(user); // "Alice (25) - alice@example.com"
```

Array Destructuring

```
const colors = ["red", "green", "blue", "yellow"];

const [first, second] = colors;
console.log(first); // "red"
console.log(second); // "green"

// Skip elements
const [, , third] = colors;
console.log(third); // "blue"

// With rest operator
const [primary, ...rest] = colors;
console.log(primary); // "red"
console.log(rest); // ["green", "blue", "yellow"]

// Swap variables (no temp needed!)
let a = 1, b = 2;
[a, b] = [b, a];
console.log(a, b); // 2, 1

// With default values
const [x = 10, y = 20, z = 30] = [1, 2];
console.log(x, y, z); // 1, 2, 30
```

8.4 Spread Operator (...)

The spread operator **expands** an iterable (array, string, object) into individual elements.

```
// Spreading arrays
const fruits = ["apple", "banana"];
const moreFruits = ["cherry", ...fruits, "date"];
console.log(moreFruits); // ["cherry", "apple", "banana", "date"]

// Copying arrays (shallow)
const original = [1, 2, 3];
const copy = [...original];
copy.push(4);
console.log(original); // [1, 2, 3] – not changed!
console.log(copy);     // [1, 2, 3, 4]

// Merging arrays
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2]; // [1, 2, 3, 4]

// Spreading objects
const defaults = { theme: "dark", fontSize: 14, language: "en" };
const userPrefs = { fontSize: 18, language: "es" };
const settings = { ...defaults, ...userPrefs };
console.log(settings); // { theme: "dark", fontSize: 18, language: "es" }
// userPrefs overrides defaults for matching keys

// Spreading strings
const letters = [..."hello"];
console.log(letters); // ["h", "e", "l", "l", "o"]

// Function arguments
const numbers = [5, 2, 9, 1, 7];
console.log(Math.max(...numbers)); // 9
```

8.5 Rest Operator (...)

The rest operator **collects** multiple elements into an array or object. Same syntax as spread (...), but used in **different positions**.

```
// In function parameters – collect remaining args
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
console.log(sum(1, 2, 3, 4)); // 10
console.log(sum(10, 20));     // 30

// With other parameters
function logInfo(name, age, ...hobbies) {
  console.log(`${name}, ${age}`);
  console.log("Hobbies:", hobbies);
}
```

```
logInfo("Alice", 25, "reading", "coding", "hiking");
// "Alice, 25"
// "Hobbies: ['reading', 'coding', 'hiking']"

// In destructuring – collect remaining properties
const { a, b, ...remaining } = { a: 1, b: 2, c: 3, d: 4 };
console.log(a);           // 1
console.log(remaining); // { c: 3, d: 4 }
```

Spread vs Rest Cheat Sheet

```
// SPREAD: unpacks/expands (used on the right side of =)
const newArr = [...oldArr];    // expand array
const newObj = { ...oldObj };  // expand object

// REST: packs/collects (used on the left side or in parameters)
const [first, ...rest] = arr;  // collect remaining
function fn(...args) { /* args is an array */ }
```

8.6 Default Parameters

Set **fallback values** for function parameters.

```
function greet(name = "Guest", greeting = "Hello") {
  console.log(`${greeting}, ${name}!`);
}

greet();           // "Hello, Guest!"
greet("Alice");    // "Hello, Alice!"
greet("Bob", "Hey"); // "Hey, Bob!"
greet(undefined, "Howdy"); // "Howdy, Guest!" – undefined triggers default

// With expressions
function createUser(name, role = "user", createdAt = new Date()) {
  return { name, role, createdAt };
}

// Using previous parameters
function calculateArea(width, height = width) {
  return width * height; // square by default
}

console.log(calculateArea(5)); // 25 (square)
console.log(calculateArea(5, 3)); // 15 (rectangle)
```

8.7 Optional Chaining (?.)

Safely access deeply nested properties **without checking each level** for `null` or `undefined`.

```
const user = {
  name: "Alice",
  address: {
    city: "New York"
  }
};

// Without optional chaining (tedious and error-prone)
const zip1 = user && user.address && user.address.zipCode;
console.log(zip1); // undefined

// With optional chaining (clean!)
const zip2 = user?.address?.zipCode;
console.log(zip2); // undefined (no error!)

// Works with methods
const result = user?.getProfile?.();
console.log(result); // undefined (no error, even though getProfile doesn't exist)

// Works with arrays
const users = [{ name: "Alice" }];
console.log(users?.[0]?.name); // "Alice"
console.log(users?.[5]?.name); // undefined (no error)

// Real-world API response handling
const apiResponse = {
  data: {
    users: [
      { name: "Alice", preferences: { theme: "dark" } },
      { name: "Bob" }
    ]
  }
};

const theme = apiResponse?.data?.users?.[0]?.preferences?.theme;
console.log(theme); // "dark"

const bobTheme = apiResponse?.data?.users?.[1]?.preferences?.theme;
console.log(bobTheme); // undefined – Bob has no preferences
```

8.8 Nullish Coalescing (??)

Returns the right-hand value only when the left-hand value is `null` or `undefined` (NOT for `0`, `""`, or `false`).

```
// Problem with || (logical OR) – treats all falsy values as "missing"
const count1 = 0 || 10; // 10 ✗ – 0 is a valid count!
const name1 = "" || "Guest"; // "Guest" ✗ – empty string might be intentional
```

```
const active1 = false || true; // true ✗ - false might be intentional

// Solution with ?? (nullish coalescing)
const count2 = 0 ?? 10; // 0 ☑ - 0 is NOT null/undefined
const name2 = "" ?? "Guest"; // "" ☑ - empty string is NOT null/undefined
const active2 = false ?? true; // false ☑ - false is NOT null/undefined

const missing1 = null ?? "default"; // "default" ☑
const missing2 = undefined ?? "default"; // "default" ☑

// Combined with optional chaining
const user = { settings: {} };
const theme = user?.settings?.theme ?? "light";
console.log(theme); // "light" - settings.theme is undefined
```

8.9 Modules (import/export)

ES6 Modules let you split code into **separate files** and share functionality between them.

Named Exports

```
// math.js - exporting
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

export const PI = 3.14159;
```

```
// app.js - importing
import { add, subtract, PI } from "./math.js";

console.log(add(2, 3)); // 5
console.log(subtract(10, 4)); // 6
console.log(PI); // 3.14159

// Renaming imports
import { add as sum } from "./math.js";
console.log(sum(2, 3)); // 5

// Import everything
import * as MathUtils from "./math.js";
console.log(MathUtils.add(2, 3)); // 5
console.log(MathUtils.PI); // 3.14159
```

Default Exports

Each file can have **one** default export.

```
// logger.js
export default function log(message) {
  console.log(`[LOG] ${message}`);
}

// Also works with classes
// UserService.js
export default class UserService {
  getUser(id) { /* ... */ }
}
```

```
// app.js – importing default (no curly braces, any name works)
import log from "./logger.js";
import UserService from "./UserService.js";

log("Hello!"); // "[LOG] Hello!"
```

Mixed Exports

```
// api.js
export default class API {
  constructor(baseUrl) {
    this.baseUrl = baseUrl;
  }
}

export function createAPI(url) {
  return new API(url);
}

export const VERSION = "1.0.0";
```

```
// app.js
import API, { createAPI, VERSION } from "./api.js";
```

Re-exporting (Barrel Files)

```
// utils/index.js – re-export everything from sub-modules
export { add, subtract } from "./math.js";
```

```
export { default as Logger } from "./logger.js";
export { formatDate, formatCurrency } from "./formatters.js";
```

```
// app.js – clean single import
import { add, Logger, formatDate } from "./utils/index.js";
```

 Exercise — ES6+

- 1. Rewrite this code using destructuring, template literals, and arrow functions:

```
function getInfo(user) {
  var name = user.name;
  var age = user.age;
  return "Name: " + name + ", Age: " + age;
}
```

- 2. Use optional chaining and nullish coalescing to safely access `user.profile.avatar ?? "default.png"`
- 3. Create two files — one that exports a function and one that imports and uses it.

 Part 8 Summary

Concept	Key Point
let/const	Block-scoped, prefer const by default
Template literals	Backticks with \${} for expressions
Destructuring	Unpack arrays/objects into variables
Spread (...)	Expand arrays/objects into individual elements
Rest (...)	Collect remaining elements into array/object
Default parameters	Fallback values when arguments are missing
Optional chaining	?. safely access nested properties
Nullish coalescing	?? fallback only for null/undefined
Modules	import/export to split and share code

Part 9 — Real World Examples

9.1 Fetching API Data Using Async/Await

```
// Simple API fetch
async function getUser(userId) {
  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/users/${userId}`
    );

    if (!response.ok) {
      throw new Error(`HTTP error! Status: ${response.status}`);
    }

    const user = await response.json();
    console.log("User:", user.name);
    console.log("Email:", user.email);
    console.log("City:", user.address.city);
    return user;
  } catch (error) {
    console.error("Failed to fetch user:", error.message);
    return null;
  }
}

getUser(1);
// User: Leanne Graham
// Email: Sincere@april.biz
// City: Gwenborough
```

Building a Reusable API Helper

```
async function apiRequest(url, options = {}) {
  const defaultOptions = {
    headers: {
      "Content-Type": "application/json"
    }
  };

  const config = { ...defaultOptions, ...options };

  try {
    const response = await fetch(url, config);

    if (!response.ok) {
      const errorBody = await response.text();
      throw new Error(`HTTP ${response.status}: ${errorBody}`);
    }

    return await response.json();
  } catch (error) {
    if (error.name === "TypeError") {
      throw new Error("Network error – check your internet connection");
    }
  }
}
```

```
    }
    throw error;
  }
}

// GET request
async function getUsers() {
  const users = await apiRequest("https://jsonplaceholder.typicode.com/users");
  console.log(`Found ${users.length} users`);
  return users;
}

// POST request
async function createPost(title, body, userId) {
  const newPost = await apiRequest(
    "https://jsonplaceholder.typicode.com/posts",
    {
      method: "POST",
      body: JSON.stringify({ title, body, userId })
    }
  );
  console.log("Created post with ID:", newPost.id);
  return newPost;
}
```

9.2 Handling Multiple Async Requests

When you need to fetch data from **multiple sources**:

Sequential (One After Another)

```
// ✗ Slow – each request waits for the previous one
async function getDataSequentially() {
  console.time("Sequential");

  const users = await fetch("https://jsonplaceholder.typicode.com/users")
    .then((r) => r.json());
  const posts = await fetch("https://jsonplaceholder.typicode.com/posts")
    .then((r) => r.json());
  const comments = await fetch("https://jsonplaceholder.typicode.com/comments")
    .then((r) => r.json());

  console.timeEnd("Sequential"); // ~900ms (300ms × 3)
  return { users, posts, comments };
}
```

Parallel (All At Once) — BETTER!

```
// ☒ Fast – all requests start at the same time
async function getDataParallel() {
  console.time("Parallel");

  const [users, posts, comments] = await Promise.all([
    fetch("https://jsonplaceholder.typicode.com/users").then((r) => r.json()),
    fetch("https://jsonplaceholder.typicode.com/posts").then((r) => r.json()),
    fetch("https://jsonplaceholder.typicode.com/comments").then((r) => r.json())
  ]);

  console.timeEnd("Parallel"); // ~300ms (all run simultaneously)
  return { users, posts, comments };
}
```

Parallel with Error Handling (Most Robust)

```
async function getDataSafely() {
  const results = await Promise.allSettled([
    fetch("https://jsonplaceholder.typicode.com/users").then((r) => r.json()),
    fetch("https://invalid-url.example.com/data").then((r) => r.json()),
    fetch("https://jsonplaceholder.typicode.com/posts").then((r) => r.json())
  ]);

  results.forEach(function(result, index) {
    if (result.status === "fulfilled") {
      console.log(`Request ${index + 1}: ☒ Got ${result.value.length} items`);
    } else {
      console.log(`Request ${index + 1}: ☒ Failed – ${result.reason}`);
    }
  });

  // Extract only successful results
  const successfulData = results
    .filter((r) => r.status === "fulfilled")
    .map((r) => r.value);

  return successfulData;
}
```

9.3 Debouncing a Search Input

A real-world implementation of a search bar that only calls the API after the user stops typing.

```
// debounce utility
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
```

```

    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}

// API search function
async function searchUsers(query) {
  if (!query.trim()) {
    console.log("Search cleared");
    return [];
  }

  console.log(`🔍 Searching for: "${query}"`);

  try {
    const response = await fetch(
      `https://jsonplaceholder.typicode.com/users?name_like=${query}`
    );
    const users = await response.json();
    console.log(`Found ${users.length} results`);
    return users;
  } catch (error) {
    console.error("Search failed:", error.message);
    return [];
  }
}

// Create debounced version – waits 500ms after last keystroke
const debouncedSearch = debounce(searchUsers, 500);

// Attach to input (in a browser environment)
// document.getElementById("search-input")
//   .addEventListener("input", function(event) {
//     debouncedSearch(event.target.value);
//   });

// Simulating rapid typing:
debouncedSearch("L");           // cancelled
debouncedSearch("Le");          // cancelled
debouncedSearch("Lea");         // cancelled
debouncedSearch("Lean");        // cancelled
debouncedSearch("Leann");       // cancelled
debouncedSearch("Leanne");      // ☑️ Runs after 500ms!

```

9.4 Building a Simple API Request System Using Promises

A mini HTTP client library using Promises and classes:

```

class HTTPClient {
  constructor(baseUrl) {

```

```
    this.baseUrl = baseUrl;
  }

  async request(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;
    const config = {
      headers: { "Content-Type": "application/json" },
      ...options
    };

    const response = await fetch(url, config);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);
    }

    return response.json();
  }

  get(endpoint) { return this.request(endpoint); }

  post(endpoint, data) {
    return this.request(endpoint, {
      method: "POST",
      body: JSON.stringify(data)
    });
  }

  put(endpoint, data) {
    return this.request(endpoint, {
      method: "PUT",
      body: JSON.stringify(data)
    });
  }

  delete(endpoint) {
    return this.request(endpoint, { method: "DELETE" });
  }
}

// Usage
const api = new HTTPClient("https://jsonplaceholder.typicode.com");

async function demo() {
  // GET all users
  const users = await api.get("/users");
  console.log("Users:", users.length);

  // GET single post
  const post = await api.get("/posts/1");
  console.log("Post:", post.title);

  // CREATE a new post
  const newPost = await api.post("/posts", {
```

```
    title: "My New Post",
    body: "This is amazing!",
    userId: 1
  });
  console.log("Created:", newPost);

  // UPDATE a post
  const updated = await api.put("/posts/1", {
    title: "Updated Title"
  });
  console.log("Updated:", updated);

  // DELETE a post
  await api.delete("/posts/1");
  console.log("Deleted successfully");
}

demo().catch(console.error);
```

 Part 9 Summary

Example	Core Concepts Used
Fetching API data	async/await, try/catch, fetch
Multiple async requests	Promise.all(), Promise.allSettled()
Debounced search	Closures, debounce, async/await
API request system	Classes, Promises, destructuring, spread

Part 10 — Common JavaScript Mistakes

Mistake 1: Misunderstanding "this"

```
// ❌ BUG: 'this' is lost when method is detached
const timer = {
  seconds: 0,
  start: function() {
    setInterval(function() {
      this.seconds++; // ❌ 'this' is NOT timer!
      console.log(this.seconds); // NaN
    }, 1000);
  }
};

// ✅ FIX 1: Use arrow function (inherits parent's 'this')
const timer1 = {
  seconds: 0,
```

```
start: function() {
  setInterval(() => {
    this.seconds++; // ☒ 'this' is timer1
    console.log(this.seconds);
  }, 1000);
}
};

// ☒ FIX 2: Save 'this' in a variable
const timer2 = {
  seconds: 0,
  start: function() {
    const self = this;
    setInterval(function() {
      self.seconds++; // ☒ using saved reference
      console.log(self.seconds);
    }, 1000);
  }
};

// ☒ FIX 3: Use bind
const timer3 = {
  seconds: 0,
  start: function() {
    setInterval(function() {
      this.seconds++;
      console.log(this.seconds);
    }.bind(this), 1000); // ☒ bind 'this' to timer3
  }
};
```

Mistake 2: Callback Hell

```
// ✗ BAD: Deeply nested callbacks (the Pyramid of Doom)
getUser(userId, function(user) {
  getProfile(user.id, function(profile) {
    getSettings(profile.id, function(settings) {
      updateUI(settings, function(result) {
        console.log("Done!", result);
      });
    });
  });
});

// ☒ GOOD: Promise chain
getUser(userId)
  .then((user) => getProfile(user.id))
  .then((profile) => getSettings(profile.id))
  .then((settings) => updateUI(settings))
  .then((result) => console.log("Done!", result))
```

```
.catch((error) => console.error("Error:", error));

// ☒ BEST: async/await
async function loadUserData(userId) {
  try {
    const user = await getUser(userId);
    const profile = await getProfile(user.id);
    const settings = await getSettings(profile.id);
    const result = await updateUI(settings);
    console.log("Done!", result);
  } catch (error) {
    console.error("Error:", error);
  }
}
```

Mistake 3: Improper Async Usage

```
// ☒ BUG: forEach does NOT wait for async operations
async function processItems(items) {
  items.forEach(async function(item) {
    await processItem(item); // This does NOT wait!
  });
  console.log("All items processed"); // ☒ Runs BEFORE items are done!
}

// ☒ FIX 1: Use for...of loop (sequential)
async function processItemsCorrect(items) {
  for (const item of items) {
    await processItem(item); // ☒ Waits for each
  }
  console.log("All items processed"); // ☒ Runs after all are done
}

// ☒ FIX 2: Use Promise.all with map (parallel)
async function processItemsParallel(items) {
  await Promise.all(items.map((item) => processItem(item)));
  console.log("All items processed"); // ☒ Runs after all are done
}
```

```
// ☒ BUG: Forgetting to await
async function getUsername() {
  const response = fetch("https://api.example.com/user"); // Missing await!
  const data = response.json(); // ☒ TypeError: response.json is not a function
  return data.name;
}

// ☒ FIX: Always await async operations
async function getUsernameFixed() {
```

```
const response = await fetch("https://api.example.com/user");
const data = await response.json();
return data.name;
}
```

Mistake 4: Scope Confusion

```
// ✗ BUG: var in loops shares the same variable
for (var i = 0; i < 5; i++) {
  setTimeout(function() {
    console.log(i); // ✗ Prints: 5, 5, 5, 5, 5
  }, 100);
}

// ✓ FIX: Use let (block-scoped)
for (let i = 0; i < 5; i++) {
  setTimeout(function() {
    console.log(i); // ✓ Prints: 0, 1, 2, 3, 4
  }, 100);
}
```

```
// ✗ BUG: Accidental global variables
function calculateTotal(items) {
  total = 0; // Missing 'let/const' – creates a global variable!
  for (item of items) { // Missing 'const' – 'item' is global too!
    total += item.price;
  }
  return total;
}

// ✓ FIX: Always declare variables
function calculateTotalFixed(items) {
  let total = 0;
  for (const item of items) {
    total += item.price;
  }
  return total;
}
```

Mistake 5: Comparing with == Instead of ===

```
// ✗ == does type coercion (surprising results)
console.log(0 == ""); // true 🧠
console.log(0 == "0"); // true 🧠
```

```
console.log("" == "0");           // false
console.log(null == undefined);  // true
console.log(false == "0");       // true 🧠

// ☑️ === checks value AND type (predictable)
console.log(0 === "");           // false ☑️
console.log(0 === "0");          // false ☑️
console.log(null === undefined); // false ☑️
console.log(false === "0");      // false ☑️
```

Rule: Always use `===` and `!==` unless you have a specific reason for `==`.

Mistake 6: Mutating Objects/Arrays Unintentionally

```
// ✗ BUG: Objects are passed by reference
const original = { name: "Alice", scores: [90, 85, 92] };
const copy = original;           // NOT a copy – same reference!
copy.name = "Bob";
console.log(original.name);      // "Bob" 🧠 – original is changed!

// ☑️ FIX: Create proper copies
const shallowCopy = { ...original };           // shallow copy
const deepCopy = JSON.parse(JSON.stringify(original)); // deep copy
const modernDeepCopy = structuredClone(original); // modern deep copy
```

Part 10 Summary

Mistake	Solution
Losing <code>this</code>	Arrow functions, <code>.bind()</code> , or save <code>self = this</code>
Callback hell	Use Promises or <code>async/await</code>
<code>forEach</code> with <code>async</code>	Use <code>for...of</code> or <code>Promise.all + map</code>
Missing <code>await</code>	Always <code>await</code> <code>async</code> calls
<code>var</code> scope issues	Use <code>let</code> or <code>const</code>
<code>==</code> vs <code>===</code>	Always use <code>===</code>
Object mutation	Use spread or <code>structuredClone()</code> for copies

Part 11 — Practice Section

11.1 Conceptual Questions (20 Questions)

1. What is the difference between the Call Stack and the Event Loop?
 2. Explain why JavaScript is called "single-threaded" and how it still handles asynchronous operations.
 3. What happens during the "creation phase" of an Execution Context?
 4. What is the Temporal Dead Zone? Give an example where it would cause an error.
 5. Explain the difference between lexical scope and dynamic scope. Which one does JavaScript use?
 6. What is a closure? Write a one-sentence explanation and give a practical use case.
 7. How does `this` behave differently in arrow functions compared to regular functions?
 8. What is the difference between `call()`, `apply()`, and `bind()`?
 9. Explain the difference between the Microtask Queue and the Callback Queue (Macrotask Queue). Which has higher priority?
 10. What are the three states of a Promise? Can a Promise go from "fulfilled" to "rejected"?
 11. What is the difference between `Promise.all()` and `Promise.allSettled()`?
 12. Why can't you use `await` outside of an `async` function? (Note: top-level `await` is available in ES modules.)
 13. What is currying and why would you use it?
 14. Explain the difference between debouncing and throttling. When would you use each?
 15. What is memoization and what problem does it solve?
 16. What is the Module Pattern and why was it used before ES6 modules?
 17. Explain the prototype chain. What happens when you access a property that doesn't exist on an object?
 18. Are JavaScript classes fundamentally different from constructor functions? Explain.
 19. What is the difference between `??` (nullish coalescing) and `||` (logical OR)?
 20. Why is `const obj = {}; obj.name = "Alice";` valid even though `obj` is declared with `const`?
-

11.2 Coding Exercises (10 Exercises)

Exercise 1: Closure Counter

Create a function `makeCounter()` that returns an object with `increment()`, `decrement()`, `reset()`, and `getValue()` methods. The counter value should be private.

```
const counter = makeCounter(10); // start at 10
counter.increment(); // 11
counter.increment(); // 12
counter.decrement(); // 11
```

```
counter.reset();    // 10 (back to initial)
console.log(counter.getValue()); // 10
```

Exercise 2: Promise Chain

Write a function `processOrder(orderId)` that simulates:

1. Validating the order (1 second delay)
2. Processing payment (2 seconds delay)
3. Shipping the order (1 second delay)

Each step should return a Promise. Chain them together and log each step.

Exercise 3: Async Data Fetcher

Write an async function `getUserWithPosts(userId)` that:

1. Fetches a user from `https://jsonplaceholder.typicode.com/users/{userId}`
 2. Fetches their posts from `https://jsonplaceholder.typicode.com/posts?userId={userId}`
 3. Returns an object: `{ user, posts, postCount }`
 4. Handles errors gracefully
-

Exercise 4: Custom Array Methods

Implement your own versions of `map`, `filter`, and `reduce` from scratch (don't use the built-in methods):

```
function myMap(arr, callback) { /* ... */ }
function myFilter(arr, callback) { /* ... */ }
function myReduce(arr, callback, initialValue) { /* ... */ }
```

Exercise 5: Debounce with Immediate Option

Create an improved `debounce` function that supports an `immediate` option. When `immediate` is true, the function fires on the **leading edge** (immediately on first call), then ignores subsequent calls for the delay period.

```
const debouncedFn = debounce(fn, 500, { immediate: true });
```

Exercise 6: Deep Clone

Write a function `deepClone(obj)` that creates a deep copy of any object, handling:

- Nested objects
- Arrays
- Dates
- `null` values

```
const original = {
  name: "Alice",
  scores: [90, 85],
  address: { city: "NYC" },
  createdAt: new Date()
};

const clone = deepClone(original);
clone.scores.push(100);
console.log(original.scores.length); // 2 (not affected)
```

Exercise 7: Promise.all() from Scratch

Implement your own version of `Promise.all()`:

```
function myPromiseAll(promises) {
  // Should return a Promise that:
  // - resolves with an array of all results when ALL promises resolve
  // - rejects with the first error when ANY promise rejects
}
```

Exercise 8: Event Emitter

Build a simple `EventEmitter` class with `on(event, callback)`, `emit(event, ...data)`, and `off(event, callback)` methods:

```
const emitter = new EventEmitter();
const handler = (data) => console.log("Received:", data);

emitter.on("message", handler);
emitter.emit("message", "Hello!"); // "Received: Hello!"
emitter.off("message", handler);
emitter.emit("message", "Hello!"); // (nothing happens)
```

Exercise 9: Curry Helper

Write a generic `curry` function that works with any number of arguments:

```
function multiply(a, b, c) { return a * b * c; }

const curried = curry(multiply);
console.log(curried(2)(3)(4)); // 24
console.log(curried(2, 3)(4)); // 24
console.log(curried(2)(3, 4)); // 24
console.log(curried(2, 3, 4)); // 24
```

Exercise 10: Retry with Backoff

Create an async function `retryWithBackoff(fn, maxRetries, baseDelay)` that:

- Calls `fn()` which returns a Promise
- If it fails, waits `baseDelay * 2^attemptNumber` ms before retrying
- Stops after `maxRetries` attempts
- Returns the successful result or throws the last error

```
const result = await retryWithBackoff(unreliableAPICall, 3, 1000);
// Attempt 1: fail → wait 1000ms
// Attempt 2: fail → wait 2000ms
// Attempt 3: fail → wait 4000ms
// Attempt 4: success! → returns result
```

11.3 Mini Challenges (5 Challenges)

Challenge 1: Build a Simple State Manager

Create a state management system similar to a simplified Redux:

```
const store = createStore({ count: 0, name: "App" });

store.subscribe(function(newState) {
  console.log("State changed:", newState);
});

store.dispatch({ type: "INCREMENT" });
// State changed: { count: 1, name: "App" }

store.dispatch({ type: "SET_NAME", payload: "MyApp" });
// State changed: { count: 1, name: "MyApp" }

console.log(store.getState());
// { count: 1, name: "MyApp" }
```

Requirements:

- Use closures for private state
 - Support `subscribe`, `dispatch`, `getState`
 - Implement a reducer function for state updates
-

Challenge 2: Build an Async Task Queue

Create a task queue that processes async tasks **one at a time** (concurrency = 1):

```
const queue = new TaskQueue();

queue.add(async () => { /* task 1 */ });
queue.add(async () => { /* task 2 */ });
queue.add(async () => { /* task 3 */ });

// Tasks run ONE AT A TIME in order, even though they're async
```

Bonus: Add a `concurrency` option to run N tasks simultaneously.

Challenge 3: Implement a Caching Decorator with TTL

Create a `cacheWithTTL(fn, ttlMs)` function that caches results but **expires** them after a specified time-to-live:

```
const cachedFetch = cacheWithTTL(fetchUserData, 5000); // 5 second cache

await cachedFetch(1); // 🌐 Network call
await cachedFetch(1); // 📦 From cache
// ... wait 5 seconds
await cachedFetch(1); // 🌐 Network call again (cache expired)
```

Challenge 4: Build a Pub/Sub System

Create a publish-subscribe message system:

```
const pubsub = createPubSub();

const unsubscribe = pubsub.subscribe("user:login", function(data) {
  console.log("User logged in:", data.username);
});

pubsub.publish("user:login", { username: "Alice" });
// "User logged in: Alice"

unsubscribe(); // Remove this specific subscription
```

```
pubsub.publish("user:login", { username: "Bob" });
// (nothing - unsubscribed)
```

Challenge 5: Build Promise.race() Timeout Wrapper

Create a `withTimeout(promise, ms)` function that races a Promise against a timer:

```
const result = await withTimeout(
  fetch("https://api.example.com/data"),
  5000 // 5 second timeout
);
// If fetch takes longer than 5 seconds, throw a TimeoutError
```

Final Words

Congratulations on completing this handbook! Here's a quick recap of your journey:

Part	Topic	Key Skill Gained
1	Execution Fundamentals	Understanding how JS runs internally
2	<code>this</code> & Function Methods	Controlling function context
3	Asynchronous JavaScript	Writing non-blocking code
4	Error Handling	Building robust applications
5	Advanced Functions	Writing clean, reusable code
6	JS Patterns	Structuring code professionally
7	Prototypes	Understanding inheritance
8	Modern ES6+	Using modern JavaScript features
9	Real World Examples	Applying concepts to real projects
10	Common Mistakes	Avoiding common pitfalls
11	Practice	Reinforcing everything through coding

Remember: Reading is not enough. **Practice the exercises.** Write code every day. Build projects. Break things and fix them. That's how you truly learn JavaScript.

Happy Coding! 